

Academic Record Helper

Project Report

Submitted for: CSE1021

Faculty: G. Prabu Kannan

Student: Abir Singh Pawar

Registration No.: 26BAI10239

Submitted via: VITyarthi - Build Your Own Project

2. Introduction

For my VITyarthi project I made Academic Record Helper, a terminal Python program that lets a faculty member add, view, search, update and delete student marks. It works out each student's average and the class topper on its own, instead of the faculty having to calculate it by hand. I picked this project because I've seen my friends and faculty spend time doing exactly this manually, and I wanted to practice using classes, functions, dictionaries and file handling on something that actually solves a small real problem.

3. Problem Statement

Faculty members often keep track of student marks by hand and calculate averages and the class topper manually. This takes time and it's easy to make a small mistake - especially when two students end up with the same average and only one gets picked as the "topper" without anyone noticing the tie. There's also no easy way to keep the records afterwards, since they usually just sit on paper until the next time someone needs them.

4. Functional Requirements

#	Requirement
F1	Add a student with a name and two subject marks.
F2	View every student's marks and calculated average.
F3	Find the topper; show every student tied for the highest average.
F4	Search for a student by name, regardless of letter case.
F5	Update a student's marks.
F6	Delete a student's record.
F7	Save records automatically and reload them on the next run.

I think of these as three parts really: managing records (add/update/delete), reporting (view all/find topper), and saving the data so it isn't lost.

5. Non-Functional Requirements

Requirement	How I handled it
Usability	Simple numbered menu (1-7), plain messages, nothing to memorise.
Reliability	Bad input is caught with a message instead of crashing the program.
Maintainability	Split into 6 files by what they do, so I can fix one part at a time.
Error handling	try/except around anything that reads numbers or writes to the file.
Resource use	Just a dictionary and a small JSON file - plenty for a class list.

6. System Architecture

main.py and ui.py handle the menu and screen text. They call into StudentRepository in records.py, which does the actual work and uses calculations.py and matching.py for the average and the name matching. storage.py (with settings from config.py) handles saving everything to student_records.json.

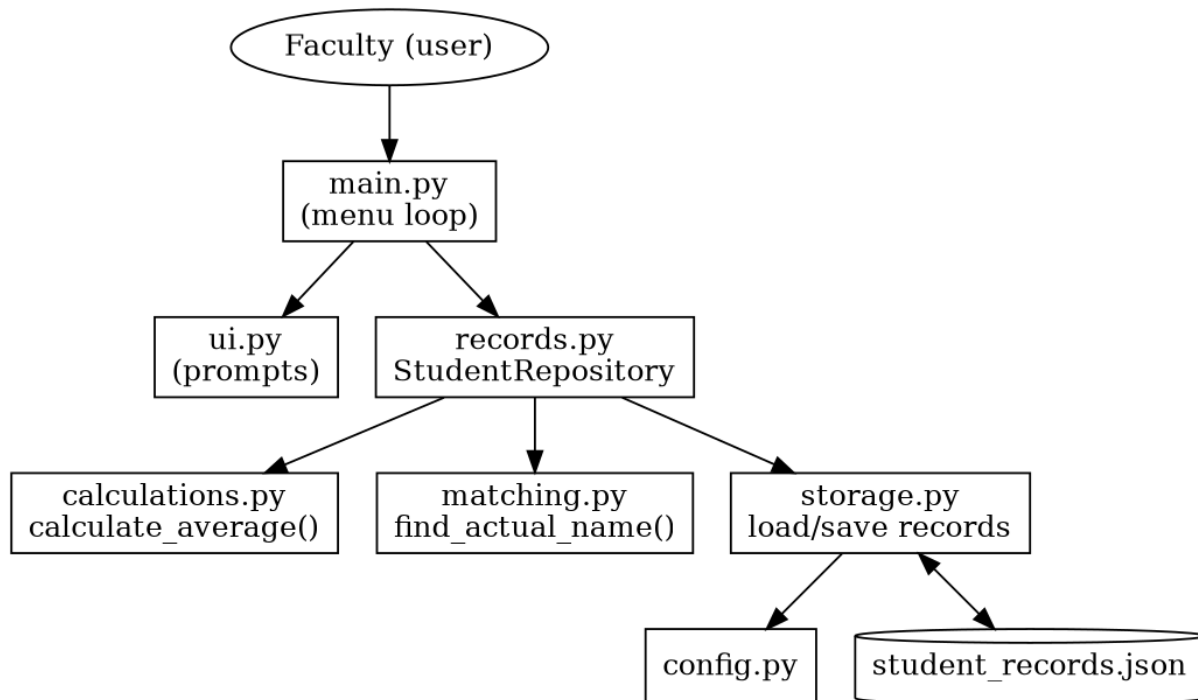


Figure 1. System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

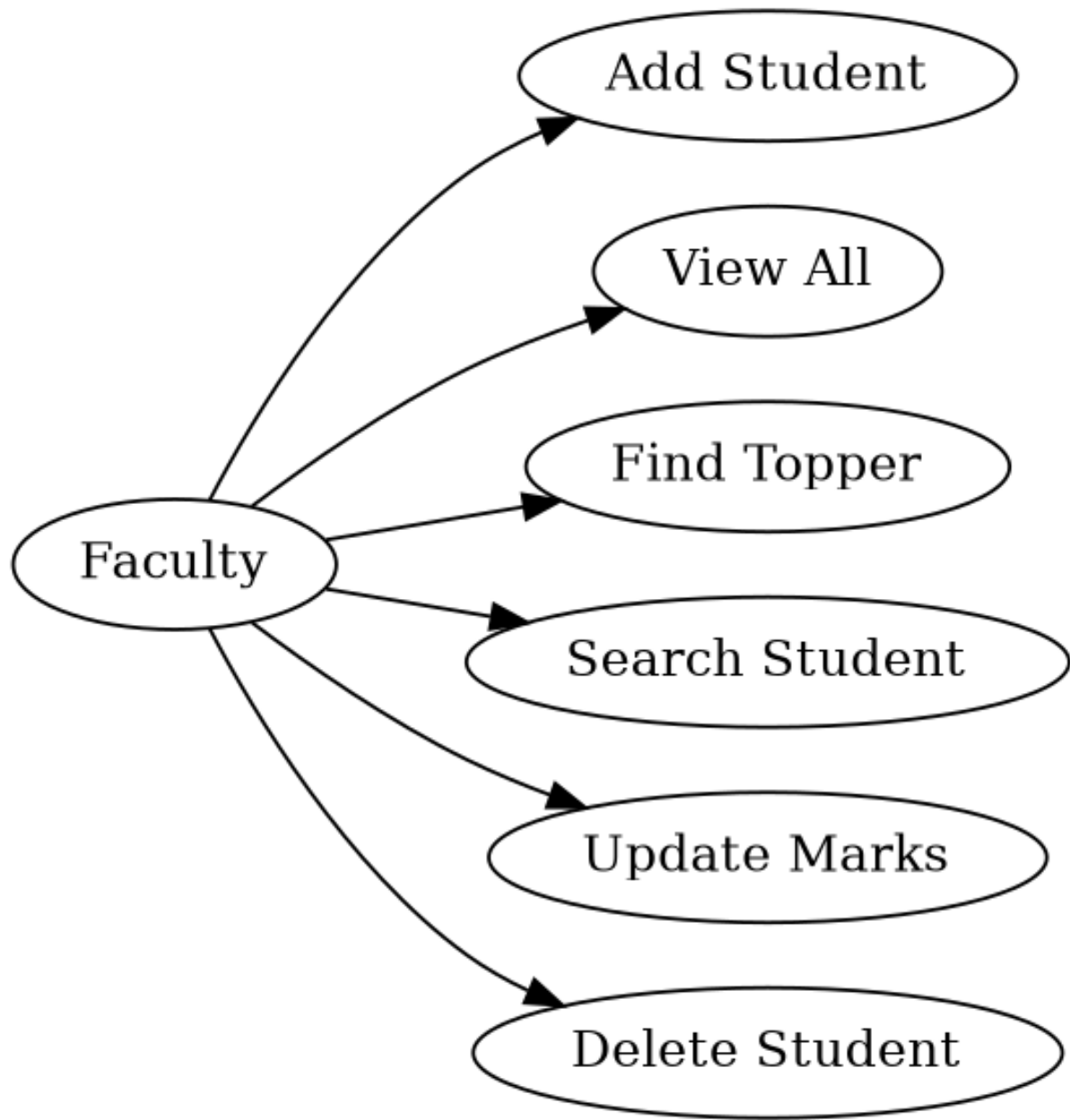


Figure 2. Use Case Diagram

7.2 Workflow Diagram

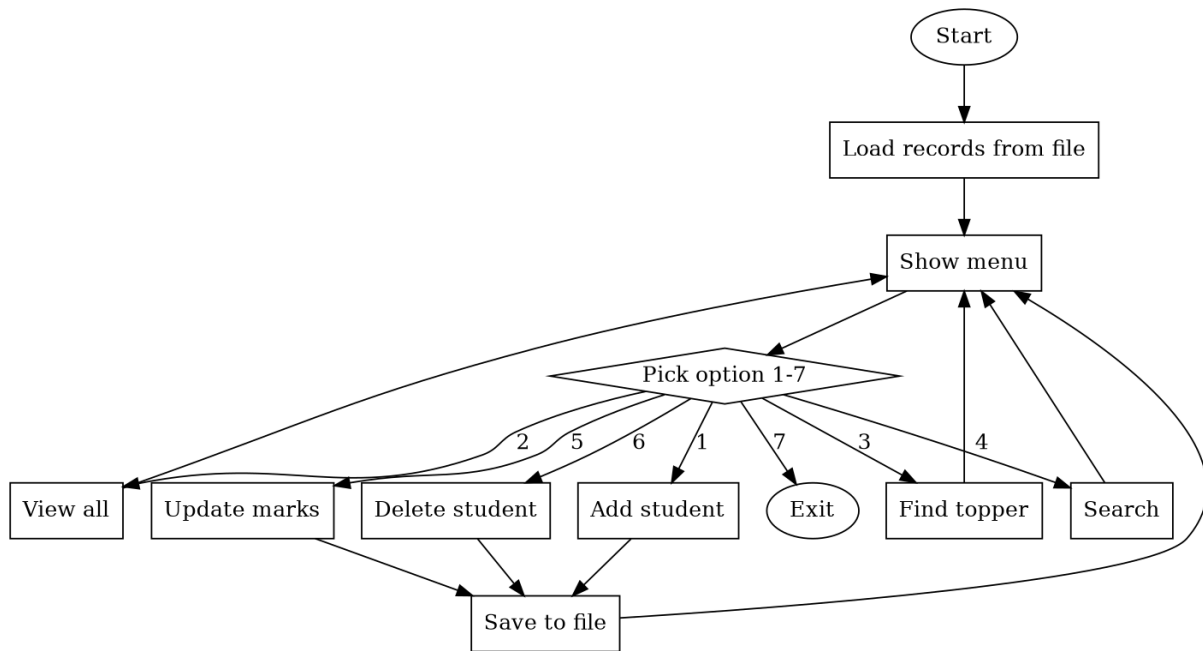


Figure 3. Process Flow / Workflow Diagram

7.3 Sequence Diagram

This traces what happens when a faculty member adds a student, from typing “1” at the menu to seeing the success message.

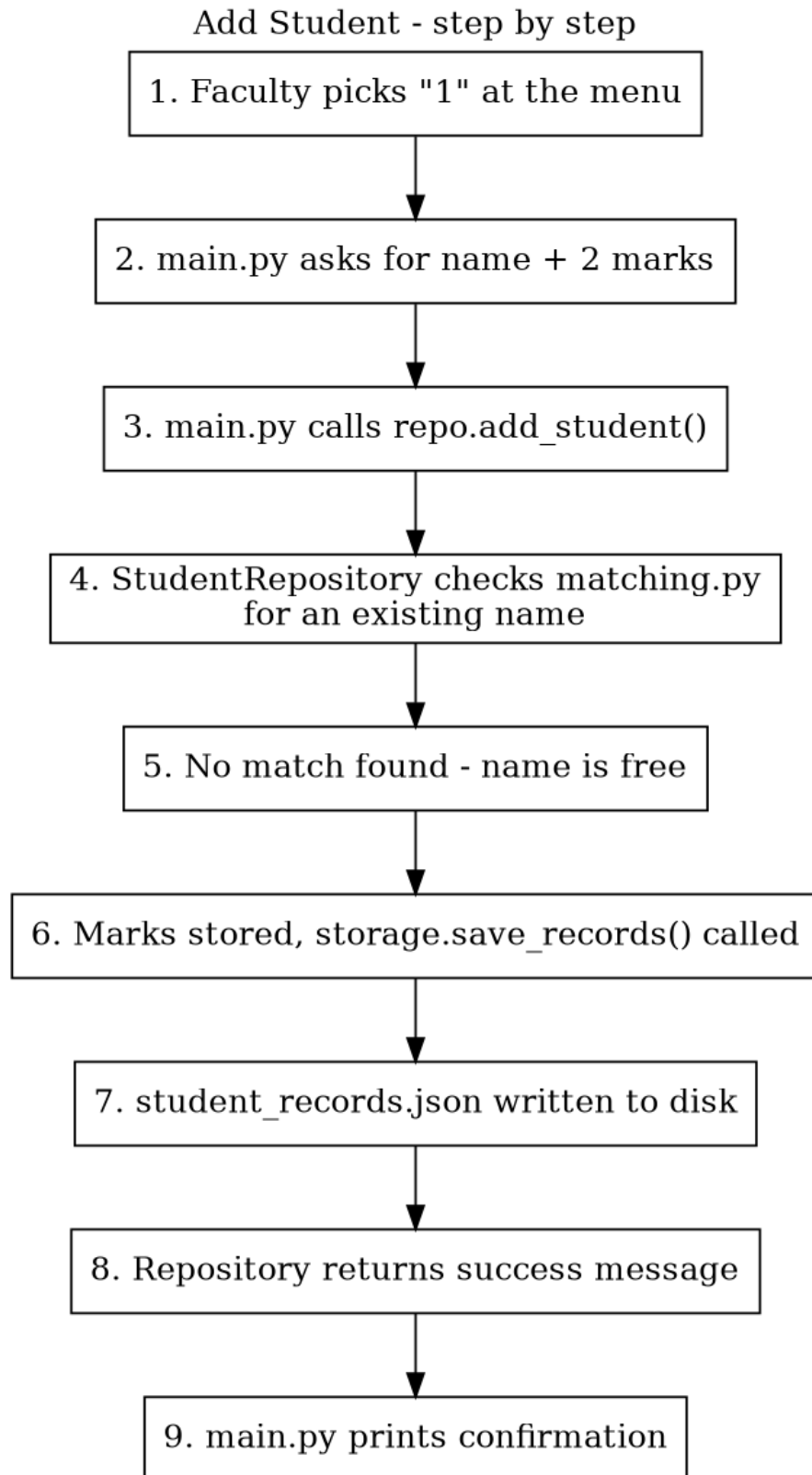


Figure 4. Sequence Diagram - Add Student

7.4 Class / Component Diagram

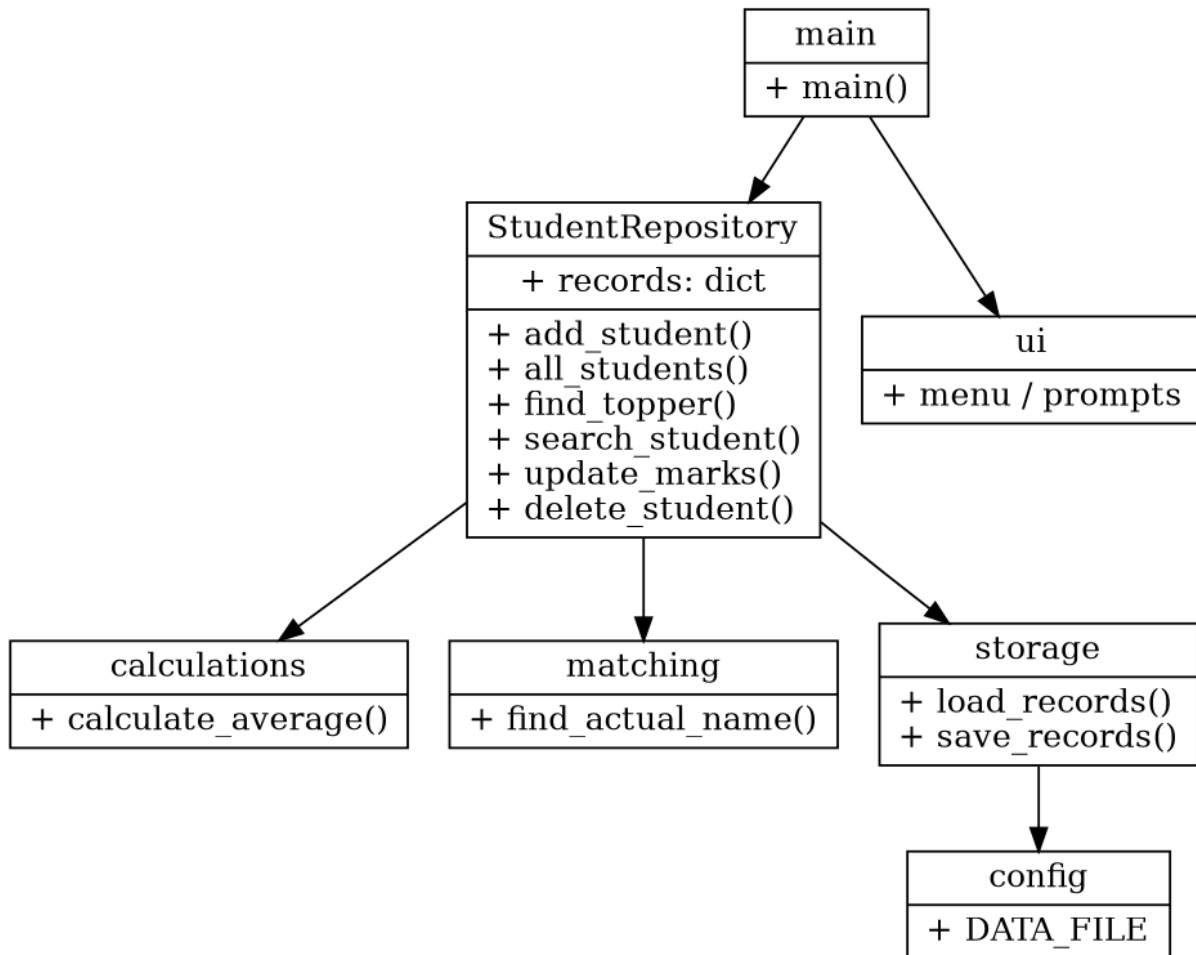


Figure 5. Class / Component Diagram

7.5 ER Diagram (Storage Design)

There is no relational database in this project; records are stored as a single JSON object where each key is a student's name and each value is a two-item list of marks. The diagram below represents this as an entity for consistency with the ER-diagram requirement.

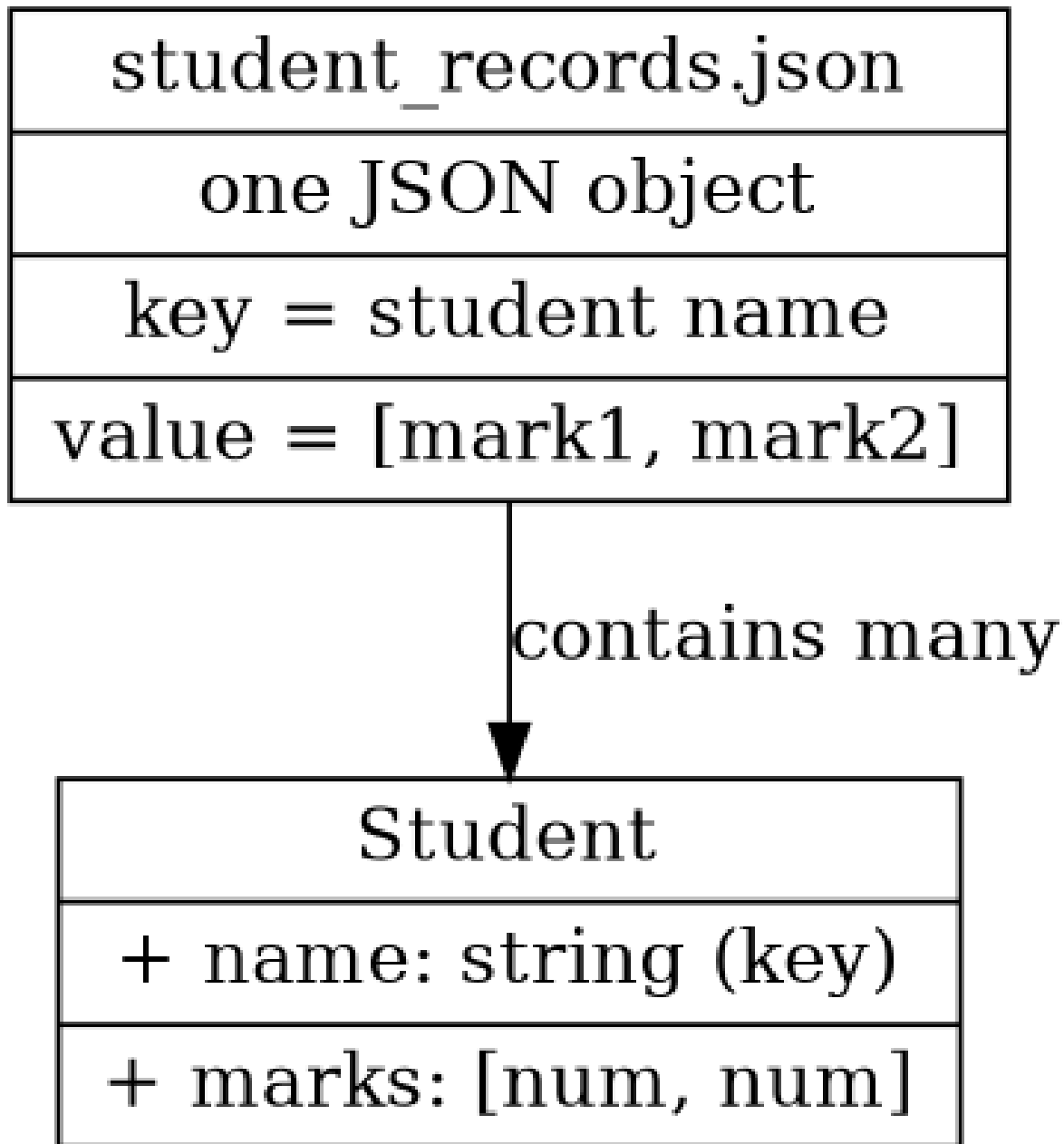


Figure 6. Storage Schema

Example of the saved JSON file:

```
{  
  "Abir": [80.0, 90.0],  
  "Bella": [85.0, 85.0]  
}
```

8. Design Decisions & Rationale

- **A class instead of separate functions:** my first version had the same rules (no duplicate names, case-insensitive search) copy-pasted in a few places. Putting it all in one StudentRepository class fixed that, and made it way easier to test.
- **One calculate_average() function:** I was repeating the same loop in the view, topper and search options before. Now there's just one place to fix if I ever change how it works.
- **JSON file instead of a database:** a class list isn't that much data, so a full database felt like overkill. JSON is also easy to just open and read if something looks off.
- **Showing every tied topper:** my old code just kept the first highest average it found. That quietly hides real ties, which didn't feel right for something meant to decide who the topper actually is.

9. Implementation Details

Everything is plain Python 3, no extra packages needed. I split it into seven files: main.py (runs the menu), ui.py (all the prompts), records.py (the StudentRepository class - this does the actual work), calculations.py (just the average function), matching.py (case-insensitive name lookup), storage.py (saving/loading the JSON), and config.py (just the file name setting). Any time something changes - adding, updating or deleting - it saves to the file right away so I never end up with the screen and the file showing different things.

10. Screenshots / Results

It's a terminal program so there isn't really a "screen" to screenshot - I'm showing a sample run instead. This one adds two students with the same average, checks that the topper shows both of them as tied, and checks that search still works when I type the name in a different case.

```
Enter your choice (1-7): 1
Enter student name: Abir
Enter marks for first subject: 80
Enter marks for second subject: 90
Success: Student saved!
...
Enter your choice (1-7): 1
Enter student name: Bella
Enter marks for first subject: 90
Enter marks for second subject: 80
Success: Student saved!
...
Enter your choice (1-7): 3
Top Performers (tied): Abir, Bella with average: 85.0
...
Enter your choice (1-7): 4
Enter name to search: ABIR
Found -> Name: Abir | Marks: [80.0, 90.0] | Average: 85.0
```

11. Testing Approach

I wrote tests in `tests/test_records.py` using Python's built-in `unittest`. They check `calculate_average()` on normal and equal marks, `find_actual_name()` matching regardless of case (and returning `None` when nobody matches), and every `StudentRepository` method - adding and searching, rejecting a duplicate name, finding a topper when there's a genuine tie, updating and deleting using a differently-cased name, and making sure records are still there after reloading from the file. They use a separate test file so they never touch my real records. All 10 tests pass.

12. Challenges Faced

- My first version repeated the same averaging loop in three different places, so pulling it into one function meant checking every place that used it still worked the same way afterward.
- Case-insensitive matching was trickier than I expected, because I still needed to show the name the way it was originally typed (so “abir” still shows up as “Abir”) while matching loosely for search. I ended up writing a small helper function for this instead of just lower-casing everything.
- Showing ties for the topper meant changing from tracking just one “best name so far” to tracking a whole list, and making sure the list resets properly whenever a strictly higher average shows up.

13. Learnings & Key Takeaways

Splitting the program into separate files made it a lot easier to act on feedback - I could fix the average calculation, the saving logic, and the name-matching each in their own file instead of hunting through one big script. Putting the record operations in a `StudentRepository` class also let me actually write tests for each one, instead of just running the whole menu by hand every time to check something worked.

14. Future Enhancements

- Support more than two subjects per student.
- Add a GUI instead of the terminal menu.
- Add easier/harder grading modes.
- Maybe password-protect the saved records file.

15. References

- Python docs (docs.python.org) - for the `json` and `unittest` modules.
- Graphviz docs (graphviz.org) - used this to draw the diagrams in this report.